Uninformed search II: space



Zola Mahlaza

Department of Computer Science

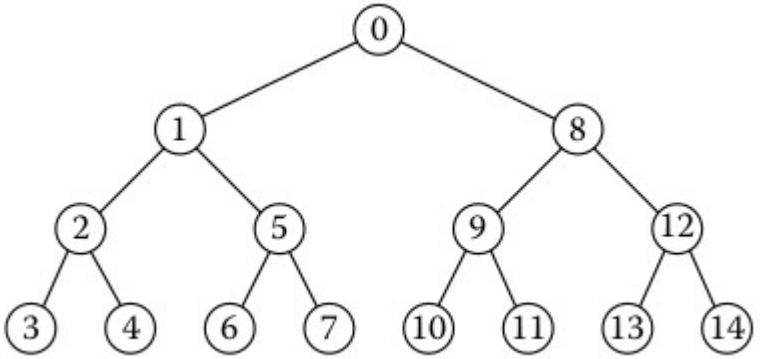
Feb 2026

Topics

- Depth-first search (DFS)
- Comparative analysis: DFS vs. BFS and Uniform-Cost Search
- Improving DFS
  - Depth limits, Iterative Deepening Search (IDS),
- Comparative summary

Depth-first search

- DFS always expands the deepest node currently in the openlist
- Use the Greedy Best-first-search framework: f(node) = −node.depth
- Why is a good idea: save on storage complexity
- A key idea: use a a last-in first-out stack for the openlist.

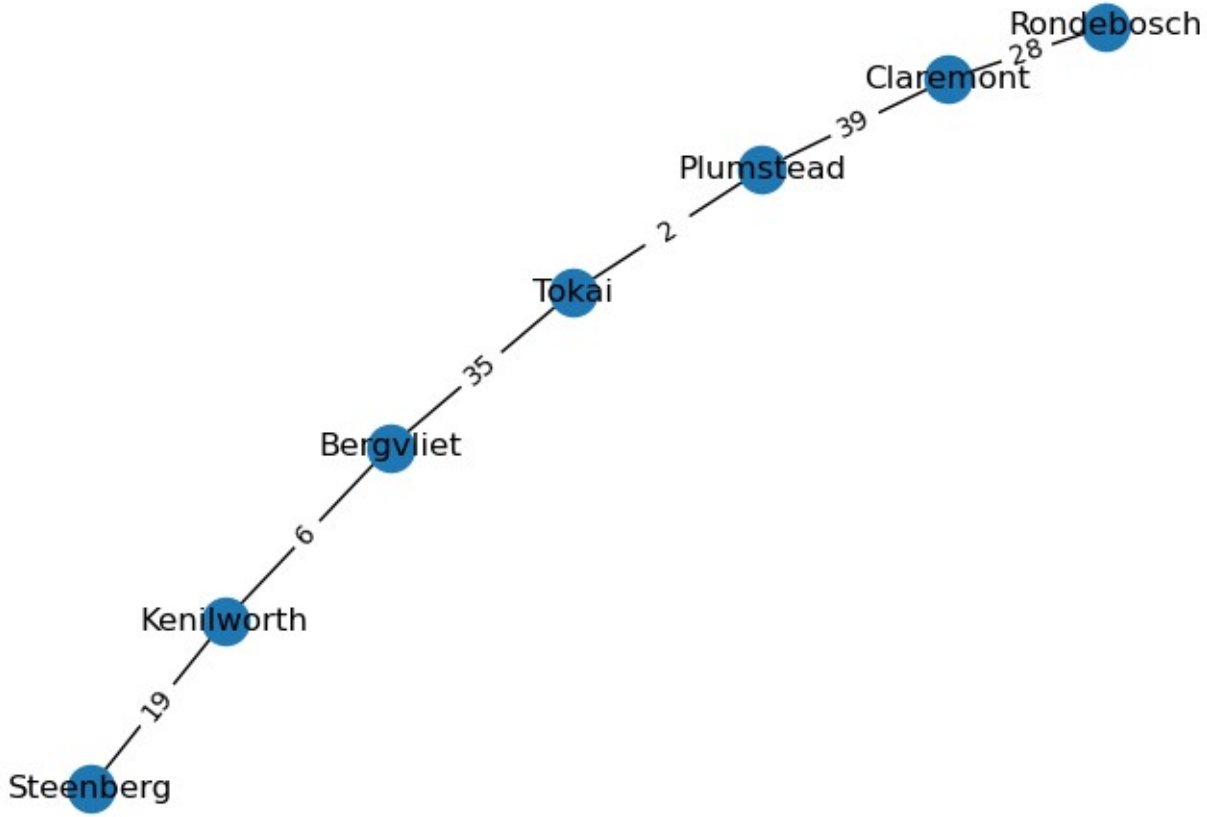


Depth-first search

Algorithm 1 Depth-first search

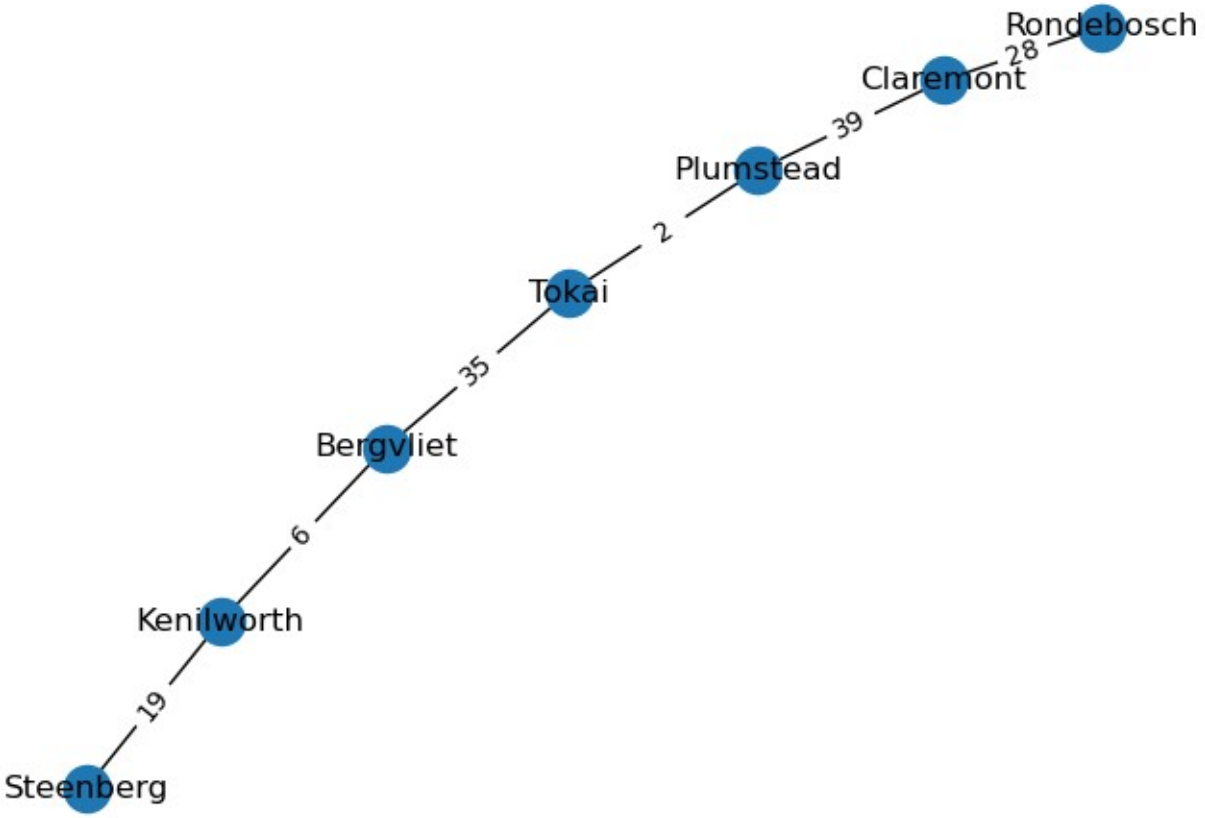
```
Initialize stack with the starting node
Mark the starting node as visited
while stack is not empty do
    Pop a node from the stack
    if node is the goal then
        Return the node
    end if
    for each child of the node do
        if child is not visited then
            Mark the child as visited
            Push the child to the stack
        end if
    end for
end while
```

Depth-first search



- Initial state: Tokai
- Goal state: Rondebosch
- Let’s consider a DFS graph search (“right-to-left”)

Depth-first search



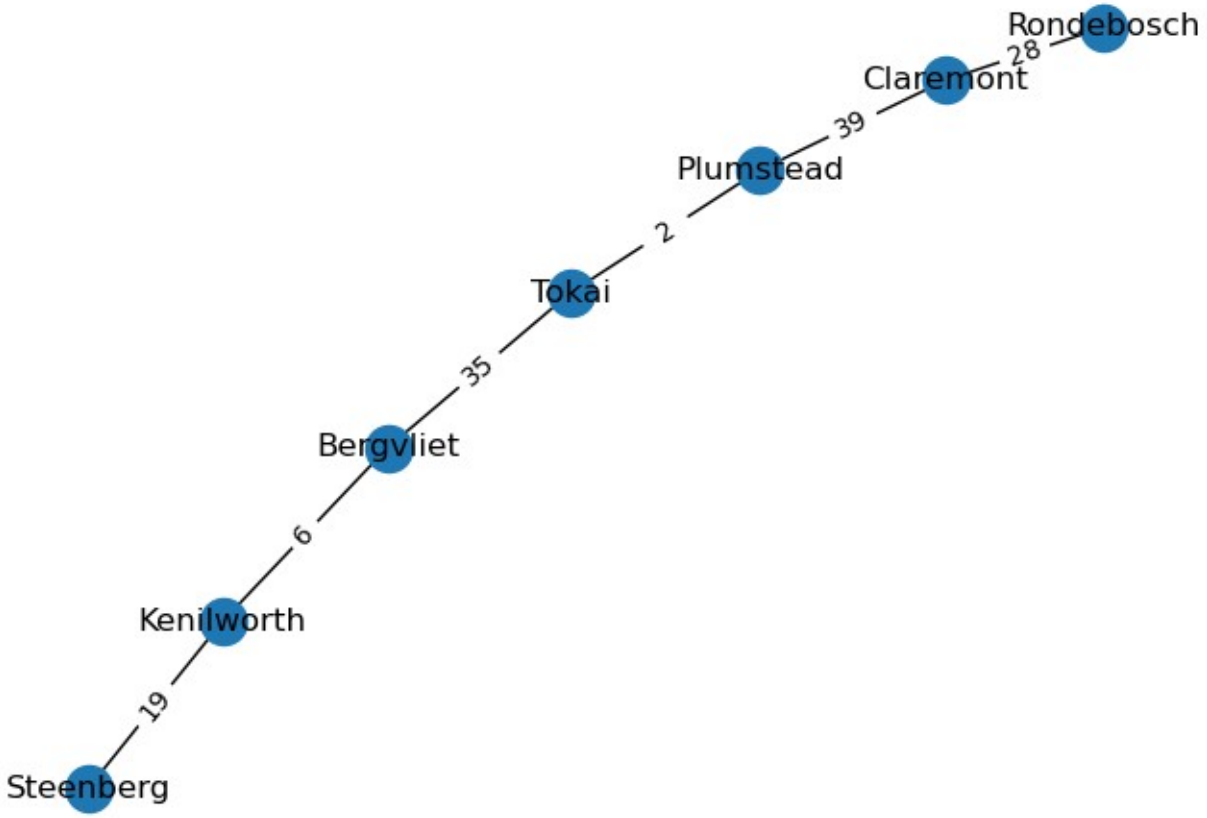


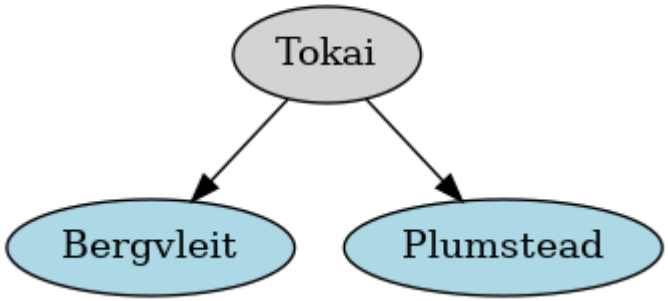
Openlist stack = {tokai}
Reached = {Tokai}

If I add Bergvleit, then Plumstead to openlist. Which will be considered first?

- Initial state: Tokai
- Goal state: Rondebosch
- Let’s consider a DFS graph search (“right-to-left”)

Depth-first search



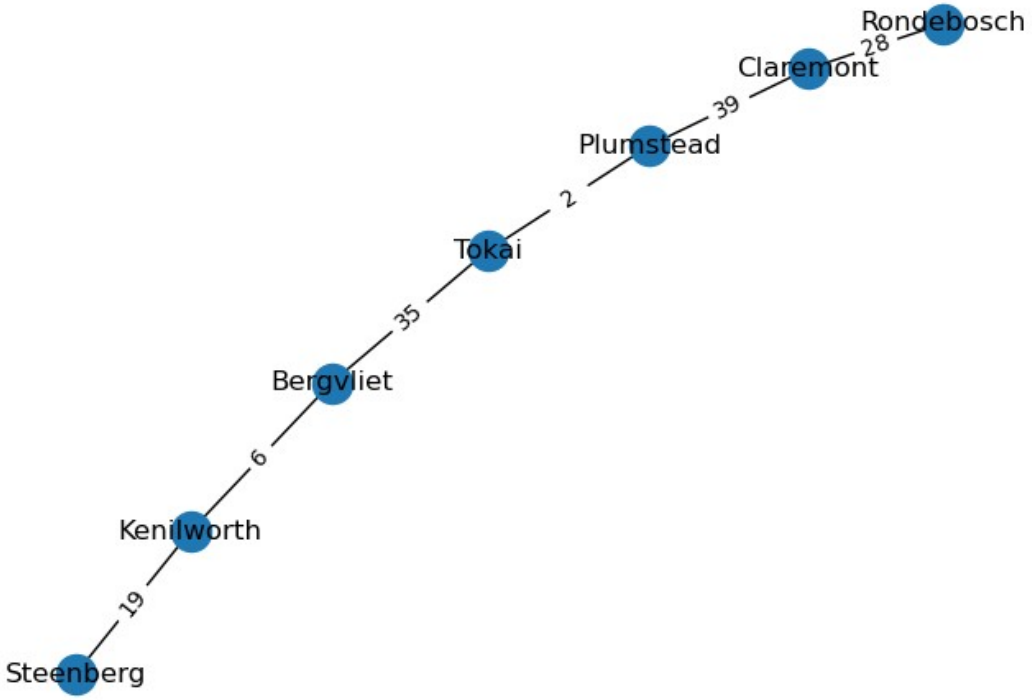


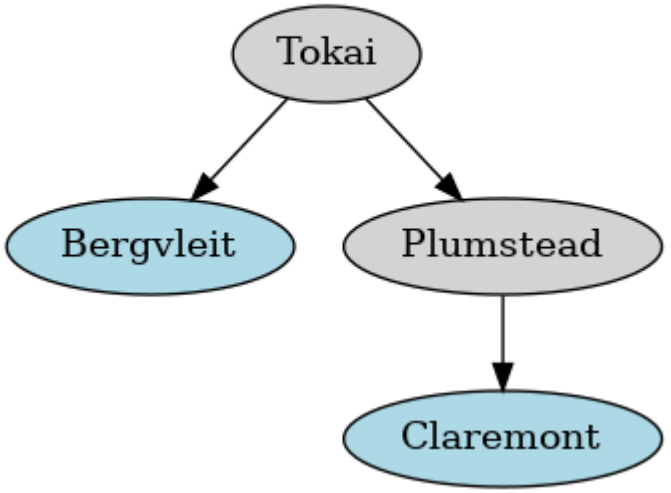
Openlist stack = {Bergvleit, Plumstead}

Reached = {Tokai, Bergvleit, Plumstead}

- Initial state: Tokai
- Goal state: Rondebosch
- Let's consider a DFS graph search ("right-to-left")

Depth-first search



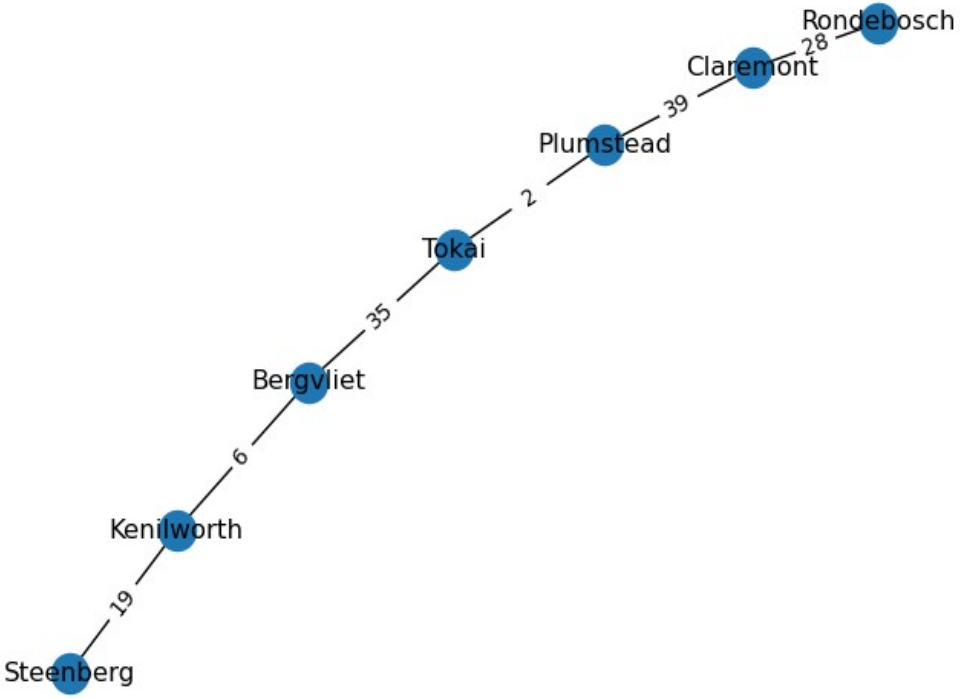


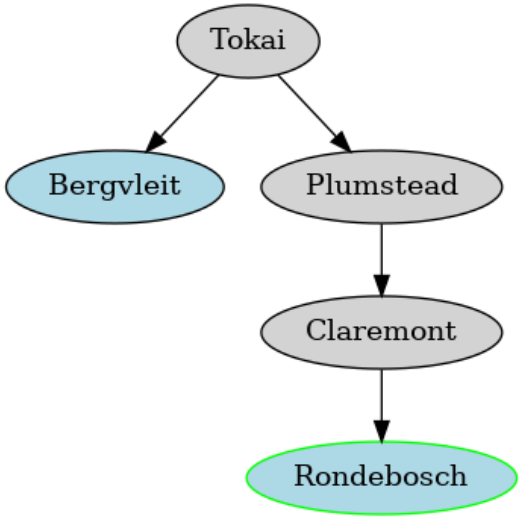
Openlist stack = {Bergvleit, Claremont}

Reached = {Tokai, Bergvleit, Plumstead, Plumstead, Claremont}

- Initial state: Tokai
- Goal state: Rondebosch
- Let’s consider a DFS graph search (“right-to-left”)

Depth-first search





Openlist stack = {Bergvleit, Rondebosch}

Reached = {Tokai, Bergvleit, Plumstead, Plumstead, Rondebosch}

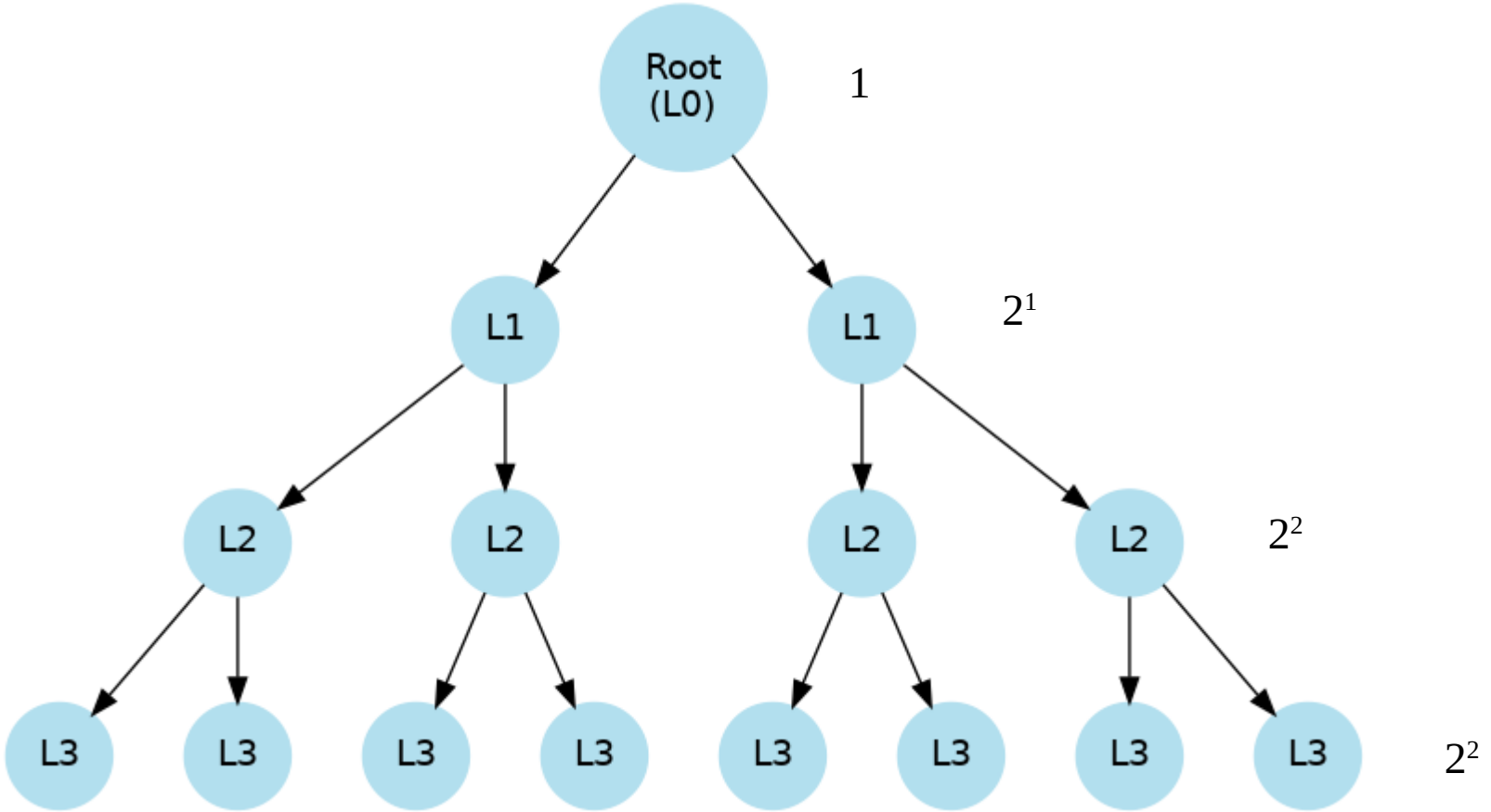
- Initial state: Tokai
- Goal state: Rondebosch
- Let’s consider a DFS graph search (“right-to-left”)

Depth-first search

- DFS is complete, as long state space is infinite and there are no cyles (tree search)
- Depth-first search is not cost-optimal
- Time complexity is $O(b^m)$ where m is max. depth
- Usually implemented as a tree-like search: susceptible to infinite loops
- Key benefit for DFS is ‘better’ memory usage when tree-like search is feasible

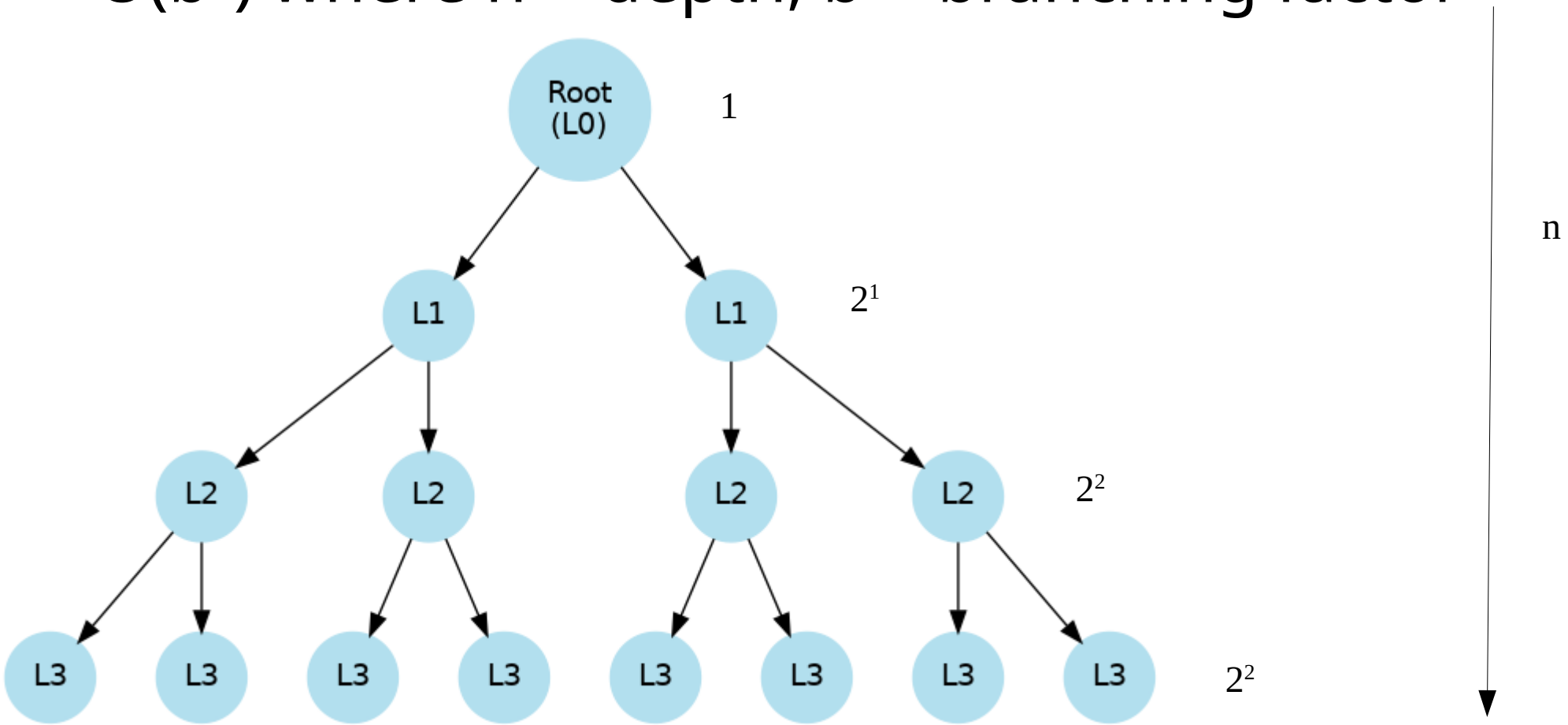
Growth of tree

- Growth of the tree is exponential
- Consider a tree with branching factor 2



Growth of tree - BFS

- Growth of the tree is exponential
- At worst case, BFS has to store all nodes
  $O(b^n)$ where n = depth, b = branching factor

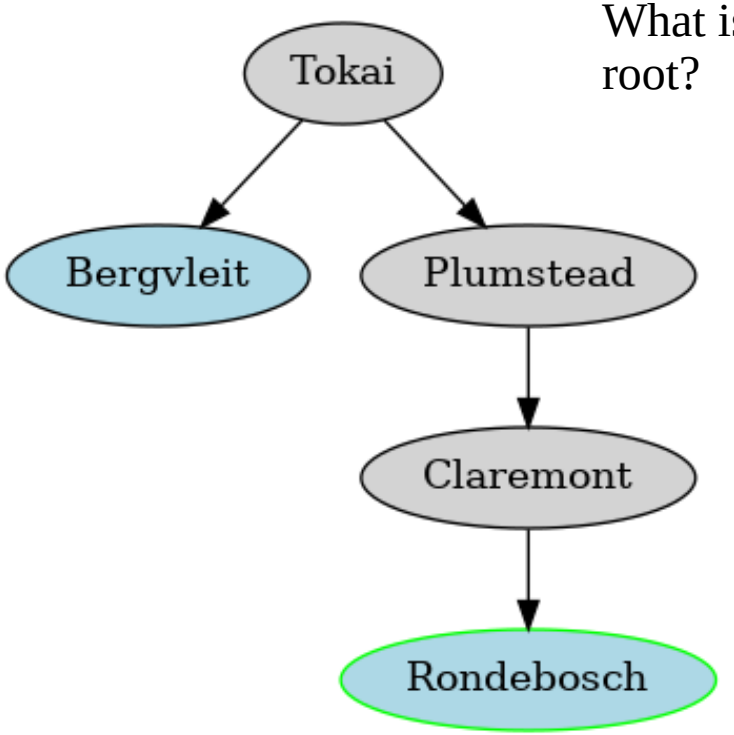


Uniform cost search

- Space complexity is $O(b^{1+\lfloor C / c \rfloor})$
- C = the cost of the optimal solution
- c = Small possible step cost
- The depth of the goal $1+\lfloor C / c \rfloor$ (in the worst case)
- If the depth is high and step cost is low, we can exponential growth

DFS – space complexity

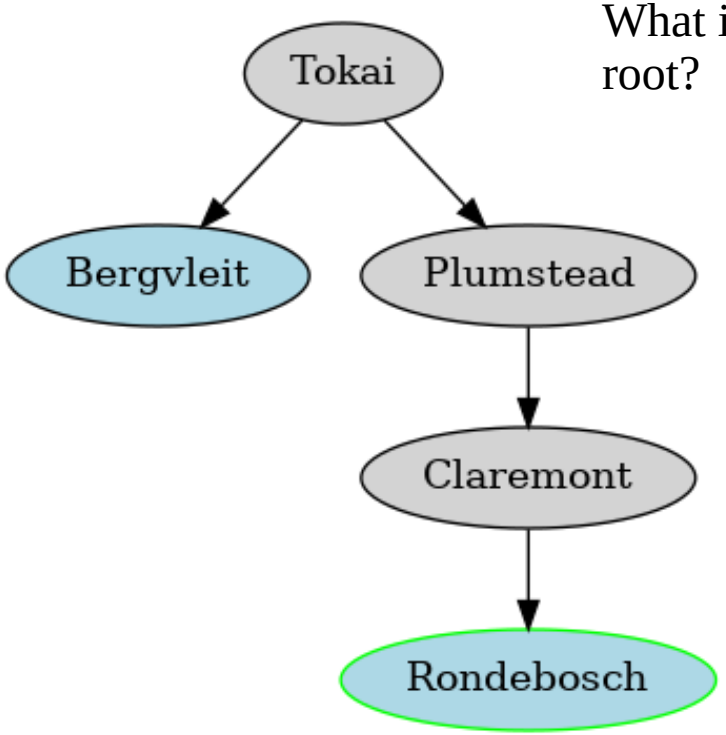
- Let n = depth, b = branching factor



What is the number of nodes at root?

DFS – space complexity

- Let n = longest depth, b = branching factor

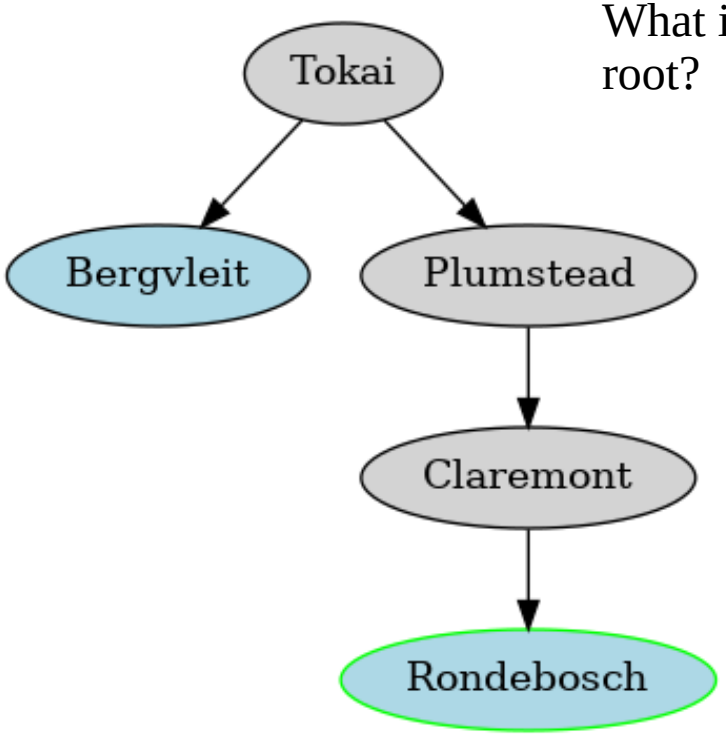


What is the number of nodes at root? 1

What is the number of nodes at level 1?

DFS – space complexity

- Let n = longest depth, b = branching factor

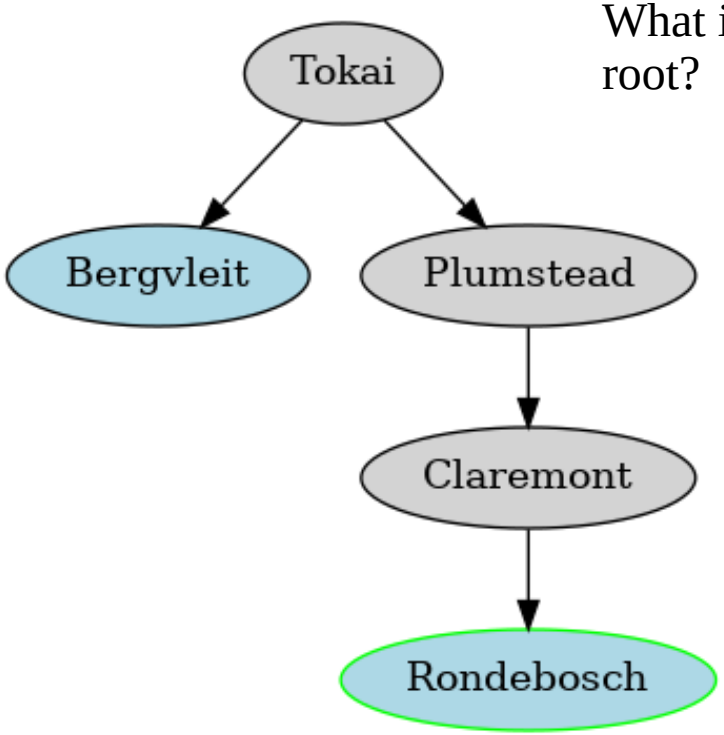


What is the number of nodes at root? 1

What is the number of nodes at level 1? b^1

DFS – space complexity

- Let n = longest depth, b = branching factor

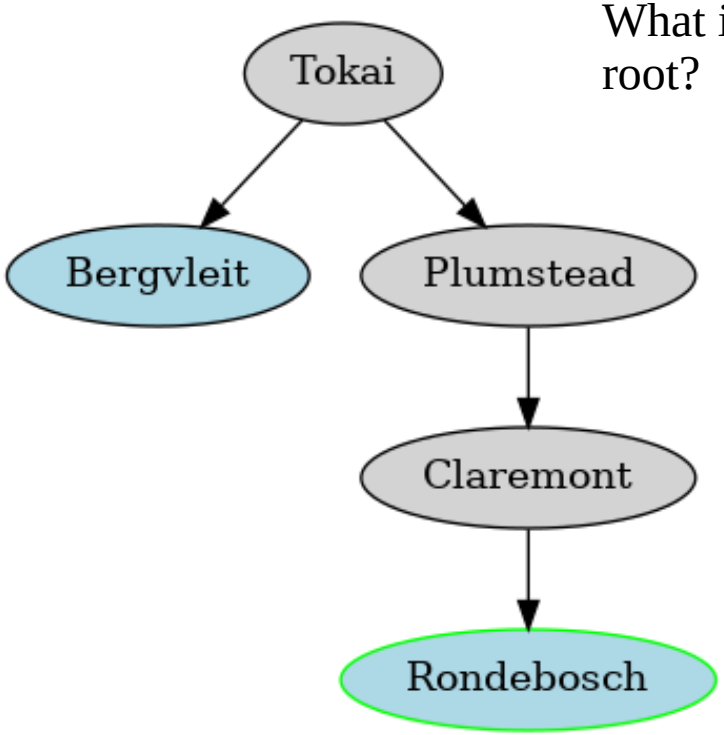


What is the number of nodes at root? 1

What is the number of nodes level 2?

DFS – space complexity

- Let n = longest depth, b = branching factor



What is the number of nodes at root? 1

What is the number of nodes at root? b and not b^i where i is the level count. Only a single node will generate children

- Storage complexity O(bn)

Improving DFS – Depth limiting

- How to avoid infinite loops in depth-first when conducting tree-like search?
- Add depth limit: and treat all nodes at depth as if they had no successors

Algorithm 1 Depth-first search (depth-limited)

```
Initialize stack with the starting node
Mark the starting node as visited
Set max depth to a predefined value
while stack is not empty do
    Pop a node from the stack
    if node is the goal then
        Return the node
    end if
    for each child of the node do                ▷ Manage depth via expand node
        if child is not visited and current depth < max depth then
            Mark the child as visited
            Push the child to the stack
        end if
    end for
end while
```

Improving DFS

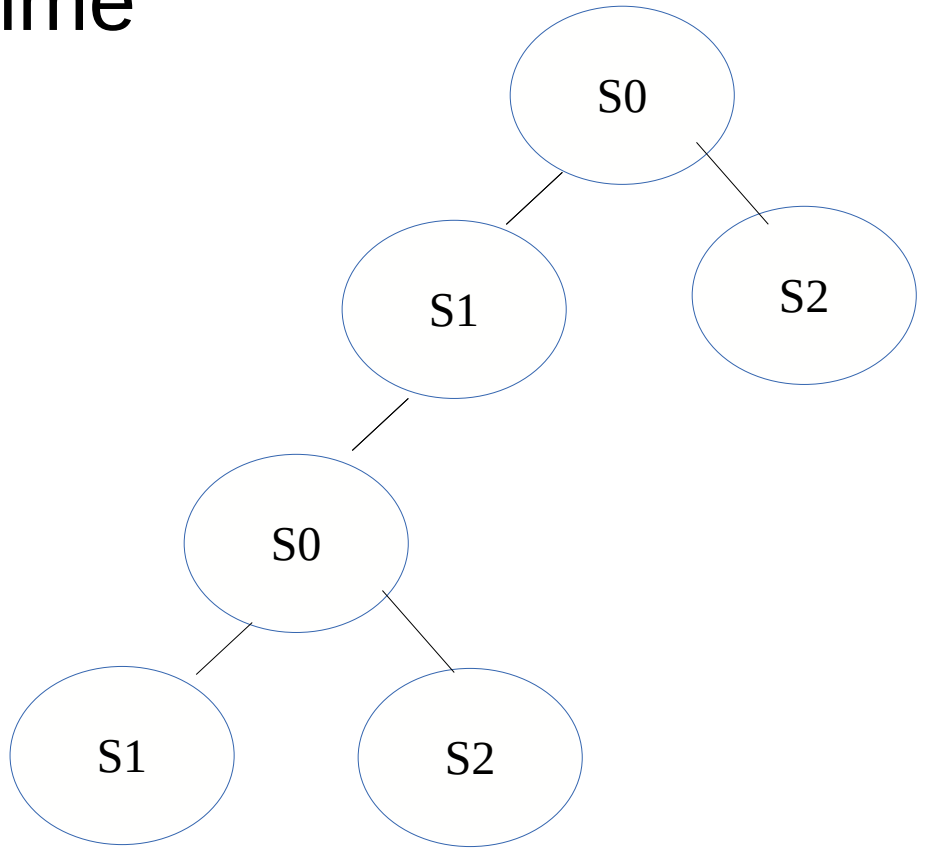
- Time complexity is still exponential $O(b^l)$
- Space complexity becomes O(bl) and l < m
  - *Search can enter cycles but no infinite loops*
- A poor choice for max depth may make the goal not reachable.
- Use problem-specific knowledge for depth limiting heuristic

Improving DFS - iterative deepening search

- DFS limits for completeness: infinite state space and graphs with cycles
- If the state space is finite, you can incrementally select the depth limit: **iterative deepening search**
  - complete on finite state spaces
- Select a starting point for max depth, increase by 1 each and search
- Either solution is found or not, or there was a cutoff
  - Outcome: by default solution not found.
  - If there solution is found then great.
  - If there was a cutoff, you cannot be sure that its not found
- ***You may enter cycles but they will be cut off.***

Improving DFS - iterative deepening search

This is not sufficient to prevent only exploring the same cycle and only increasing the depth limit by 1 each time



Improving DFS - iterative deepening search

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure
  for depth = 0 to ∞ do
    result ← DEPTH-LIMITED-SEARCH(problem, depth)
    if result ≠ cutoff then return result

function DEPTH-LIMITED-SEARCH(problem, ℓ) returns a node or failure or cutoff
  frontier ← a LIFO queue (stack) with NODE(problem.INITIAL) as an element
  result ← failure
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    if DEPTH(node) > ℓ then
      result ← cutoff
    else if not IS-CYCLE(node) do
      for each child in EXPAND(problem, node) do
        add child to frontier
  return result
```

Improving DFS - iterative deepening search

- Memory requirements are modest and no guessing about depth limit
  - O(bd) when there is a solution, or O(bm) on finite state spaces with no solution. *D= depth of solution, m = depth of entire tree*
- It follows the same top-down approach as breadth-first. However, we are saving on space.
  - You never generate all nodes per ‘level’

Improving DFS - iterative deepening search

- Repetition might seem wasteful (assume we have a solution some depth d):
  - Root will be generated d times
  - Next top nodes generated d - 1 times… $d*b^1$
  - If there is a solution at depth d, then you only generate those once ($1 * b^d$).
- Towards time complexity:
  - $d + d*b^1 + (d-1)*b1 + \ldots + 1*b^d$
  - So $O(b^d)$, same as breadth-first search

Best-First Search

	BFS	Uniform cost	DFS	DFS + depth limiting	Iterative depth limiting
Completeness	If state space is finite	If state space is finite, no 'free' actions	No	No	If state space is finite
Optimal cost	If all actions are the same	Yes	No	No	If all actions are the same
Time	$O(b^d)$	• $O(b^{1+\lfloor C / c \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor C / c \rfloor})$	$O(bm)$	$O(bl)$	$O(bd)$

Reading

- Sections 3.4.3 - 3.4.6

